





# How similar are two strings?

- Spell correction

- The user typed “graffe”

Which is closest?

- graf
    - graft
    - grail
    - giraffe

- Computational Biology

- Align two sequences of nucleotides

```
AGGCTATCACCTGACCTCCAGGCCGATGCCC
TAGCTATCACGACCGCGGTCGATTGCCCCGAC
```

- Resulting alignment:

```
-AGGCTATCACCTGACCTCCAGGCCGA--TGCCC---
TAG-CTATCAC--GACCGC--GGTCGATTGCCCCGAC
```

- Also for Machine Translation, Information Extraction, Speech Recognition



# Edit Distance

- The minimum edit distance between two strings
- Is the minimum number of editing operations
  - Insertion
  - Deletion
  - Substitution
- Needed to transform one into the other



# Minimum Edit Distance

- Two strings and their **alignment**:

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| I | N | T | E | * | N | T | I | O | N |
|   |   |   |   |   |   |   |   |   |   |
| * | E | X | E | C | U | T | I | O | N |



# Minimum Edit Distance

I N T E \* N T I O N

| | | | | | | | | |

\* E X E C U T I O N

d s s i s

- If each operation has cost of 1
  - Distance between these is 5
- If substitutions cost 2 (Levenshtein)
  - Distance between them is 8



# Alignment in Computational Biology

- Given a sequence of bases

AGGCTATCACCTGACCTCCAGGCCGATGCCC  
TAGCTATCACGACCGCGGGTCGATTGCCCCGAC

- An alignment:

–AGGCTATCACCTGACCTCCAGGCCGA–TGCCC–  
TAG–CTATCAC–GACCGC–GGTCGATTGCCCCGAC

- Given two sequences, align each letter to a letter or gap



# Other uses of Edit Distance in NLP

- Evaluating Machine Translation and speech recognition

**R** Spokesman confirms        senior government adviser was shot

**H** Spokesman said        the senior        adviser was shot dead

S

I

D

I

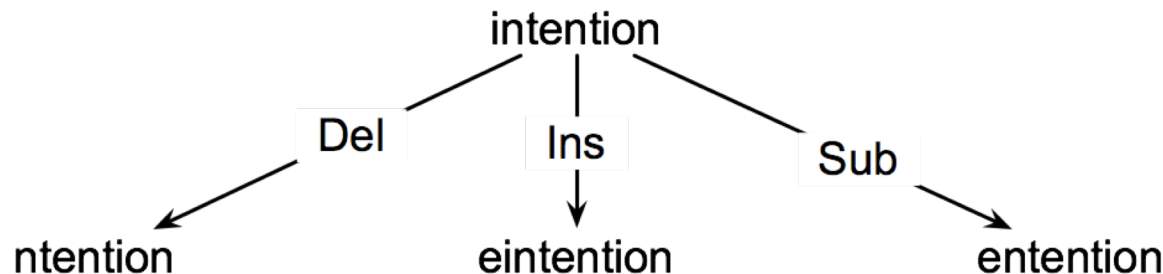
- Named Entity Extraction and Entity Coreference

- IBM Inc. announced today
- IBM profits
- Stanford President John Hennessy announced yesterday
- for Stanford University President John Hennessy



# How to find the Min Edit Distance?

- Searching for a path (sequence of edits) from the start string to the final string:
  - **Initial state:** the word we're transforming
  - **Operators:** insert, delete, substitute
  - **Goal state:** the word we're trying to get to
  - **Path cost:** what we want to minimize: the number of edits







# Minimum Edit as Search

- But the space of all edit sequences is huge!
  - We can't afford to navigate naïvely
  - Lots of distinct paths wind up at the same state.
    - We don't have to keep track of all of them
    - Just the shortest path to each of those revisited states.



# Defining Min Edit Distance

- For two strings
  - $X$  of length  $n$
  - $Y$  of length  $m$
- We define  $D(i,j)$ 
  - the edit distance between  $X[1..i]$  and  $Y[1..j]$ 
    - i.e., the first  $i$  characters of  $X$  and the first  $j$  characters of  $Y$
  - The edit distance between  $X$  and  $Y$  is thus  $D(n,m)$

[illegible]



# Dynamic Programming for Minimum Edit Distance

- **Dynamic programming:** A tabular computation of  $D(n,m)$
- Solving problems by combining solutions to subproblems.
- Bottom-up
  - We compute  $D(i,j)$  for small  $i,j$
  - And compute larger  $D(i,j)$  based on previously computed smaller values
  - i.e., compute  $D(i,j)$  for all  $i$  ( $0 < i < n$ ) and  $j$  ( $0 < j < m$ )



# Defining Min Edit Distance (Levenshtein)

- Initialization

$$D(i, 0) = i$$

$$D(0, j) = j$$

- Recurrence Relation:

For each  $i = 1 \dots M$

For each  $j = 1 \dots N$

$$D(i, j) = \min \begin{cases} D(i-1, j) + 1 \\ D(i, j-1) + 1 \\ D(i-1, j-1) + \begin{cases} 2; & \text{if } X(i) \neq Y(j) \\ 0; & \text{if } X(i) = Y(j) \end{cases} \end{cases}$$

- Termination:

$D(N, M)$  is distance



# The Edit Distance Table

|   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|
| N | 9 |   |   |   |   |   |   |   |   |   |
| O | 8 |   |   |   |   |   |   |   |   |   |
| I | 7 |   |   |   |   |   |   |   |   |   |
| T | 6 |   |   |   |   |   |   |   |   |   |
| N | 5 |   |   |   |   |   |   |   |   |   |
| E | 4 |   |   |   |   |   |   |   |   |   |
| T | 3 |   |   |   |   |   |   |   |   |   |
| N | 2 |   |   |   |   |   |   |   |   |   |
| I | 1 |   |   |   |   |   |   |   |   |   |
| # | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|   | # | E | X | E | C | U | T | I | O | N |



# The Edit Distance Table

|   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|
| N | 9 |   |   |   |   |   |   |   |   |   |
| O | 8 |   |   |   |   |   |   |   |   |   |
| I | 7 |   |   |   |   |   |   |   |   |   |
| T | 6 |   |   |   |   |   |   |   |   |   |
| N | 5 |   |   |   |   |   |   |   |   |   |
| E | 4 |   |   |   |   |   |   |   |   |   |
| T | 3 |   |   |   |   |   |   |   |   |   |
| N | 2 |   |   |   |   |   |   |   |   |   |
| I | 1 |   |   |   |   |   |   |   |   |   |
| # | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|   | # | E | X | E | C | U | T | I | O | N |

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$



# Edit Distance

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

|   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|
| N | 9 |   |   |   |   |   |   |   |   |   |
| O | 8 |   |   |   |   |   |   |   |   |   |
| I | 7 |   |   |   |   |   |   |   |   |   |
| T | 6 |   |   |   |   |   |   |   |   |   |
| N | 5 |   |   |   |   |   |   |   |   |   |
| E | 4 |   |   |   |   |   |   |   |   |   |
| T | 3 |   |   |   |   |   |   |   |   |   |
| N | 2 |   |   |   |   |   |   |   |   |   |
| I | 1 |   |   |   |   |   |   |   |   |   |
| # | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|   | # | E | X | E | C | U | T | I | O | N |





# The Edit Distance Table

|   |   |   |   |    |    |    |    |    |    |    |
|---|---|---|---|----|----|----|----|----|----|----|
| N | 9 | 8 | 9 | 10 | 11 | 12 | 11 | 10 | 9  | 8  |
| O | 8 | 7 | 8 | 9  | 10 | 11 | 10 | 9  | 8  | 9  |
| I | 7 | 6 | 7 | 8  | 9  | 10 | 9  | 8  | 9  | 10 |
| T | 6 | 5 | 6 | 7  | 8  | 9  | 8  | 9  | 10 | 11 |
| N | 5 | 4 | 5 | 6  | 7  | 8  | 9  | 10 | 11 | 10 |
| E | 4 | 3 | 4 | 5  | 6  | 7  | 8  | 9  | 10 | 9  |
| T | 3 | 4 | 5 | 6  | 7  | 8  | 7  | 8  | 9  | 8  |
| N | 2 | 3 | 4 | 5  | 6  | 7  | 8  | 7  | 8  | 7  |
| I | 1 | 2 | 3 | 4  | 5  | 6  | 7  | 6  | 7  | 8  |
| # | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
|   | # | E | X | E  | C  | U  | T  | I  | O  | N  |





# Computing alignments

- Edit distance isn't sufficient
  - We often need to **align** each character of the two strings to each other
- We do this by keeping a “backtrace”
- Every time we enter a cell, remember where we came from
- When we reach the end,
  - Trace back the path from the upper right corner to read off the alignment



# Edit Distance

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

|   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|
| N | 9 |   |   |   |   |   |   |   |   |   |
| O | 8 |   |   |   |   |   |   |   |   |   |
| I | 7 |   |   |   |   |   |   |   |   |   |
| T | 6 |   |   |   |   |   |   |   |   |   |
| N | 5 |   |   |   |   |   |   |   |   |   |
| E | 4 |   |   |   |   |   |   |   |   |   |
| T | 3 |   |   |   |   |   |   |   |   |   |
| N | 2 |   |   |   |   |   |   |   |   |   |
| I | 1 |   |   |   |   |   |   |   |   |   |
| # | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|   | # | E | X | E | C | U | T | I | O | N |



# MinEdit with Backtrace

|          |          |              |              |             |            |              |            |            |            |            |  |
|----------|----------|--------------|--------------|-------------|------------|--------------|------------|------------|------------|------------|--|
| <b>n</b> | 9        | ↓ 8          | ↙←↓ 9        | ↙←↓ 10      | ↙←↓ 11     | ↙←↓ 12       | ↓ 11       | ↓ 10       | ↓ 9        | ↙ <b>8</b> |  |
| <b>o</b> | 8        | ↓ 7          | ↙←↓ 8        | ↙←↓ 9       | ↙←↓ 10     | ↙←↓ 11       | ↓ 10       | ↓ 9        | ↙ <b>8</b> | ← 9        |  |
| <b>i</b> | 7        | ↓ 6          | ↙←↓ 7        | ↙←↓ 8       | ↙←↓ 9      | ↙←↓ 10       | ↓ 9        | ↙ <b>8</b> | ← 9        | ← 10       |  |
| <b>t</b> | 6        | ↓ 5          | ↙←↓ 6        | ↙←↓ 7       | ↙←↓ 8      | ↙←↓ 9        | ↙ <b>8</b> | ← 9        | ← 10       | ←↓ 11      |  |
| <b>n</b> | 5        | ↓ 4          | ↙←↓ 5        | ↙←↓ 6       | ↙←↓ 7      | ↙←↓ <b>8</b> | ↙←↓ 9      | ↙←↓ 10     | ↙←↓ 11     | ↙↓ 10      |  |
| <b>e</b> | 4        | ↙ 3          | ← 4          | ↙← <b>5</b> | ← <b>6</b> | ← 7          | ←↓ 8       | ↙←↓ 9      | ↙←↓ 10     | ↓ 9        |  |
| <b>t</b> | 3        | ↙←↓ 4        | ↙←↓ <b>5</b> | ↙←↓ 6       | ↙←↓ 7      | ↙←↓ 8        | ↙ 7        | ←↓ 8       | ↙←↓ 9      | ↓ 8        |  |
| <b>n</b> | 2        | ↙←↓ <b>3</b> | ↙←↓ 4        | ↙←↓ 5       | ↙←↓ 6      | ↙←↓ 7        | ↙←↓ 8      | ↓ 7        | ↙←↓ 8      | ↙ 7        |  |
| <b>i</b> | <b>1</b> | ↙←↓ 2        | ↙←↓ 3        | ↙←↓ 4       | ↙←↓ 5      | ↙←↓ 6        | ↙←↓ 7      | ↙ 6        | ← 7        | ← 8        |  |
| <b>#</b> | <b>0</b> | 1            | 2            | 3           | 4          | 5            | 6          | 7          | 8          | 9          |  |
|          | <b>#</b> | <b>e</b>     | <b>x</b>     | <b>e</b>    | <b>c</b>   | <b>u</b>     | <b>t</b>   | <b>i</b>   | <b>o</b>   | <b>n</b>   |  |



# Adding Backtrace to Minimum Edit Distance

- Base conditions:

$$D(i, 0) = i$$

$$D(0, j) = j$$

Termination:

$D(N, M)$  is distance

- Recurrence Relation:

For each  $i = 1 \dots M$

For each  $j = 1 \dots N$

$$D(i, j) = \min \begin{cases} D(i-1, j) + 1 \\ D(i, j-1) + 1 \\ D(i-1, j-1) + \end{cases}$$

deletion

insertion

$$2; \begin{cases} \text{if } X(i) \neq Y(j) \\ 0; \text{if } X(i) = Y(j) \end{cases}$$

substitution

$$\text{ptr}(i, j) = \begin{cases} \text{LEFT} \\ \text{DOWN} \\ \text{DIAG} \end{cases}$$

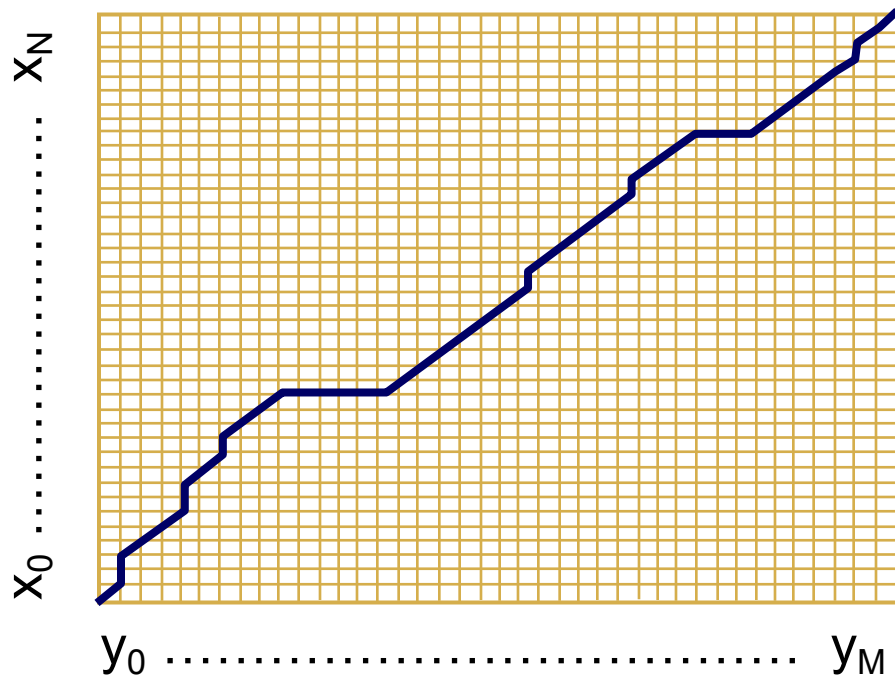
insertion

deletion

substitution



# The Distance Matrix



Every non-decreasing path

from  $(0,0)$  to  $(M, N)$

corresponds to  
an alignment  
of the two sequences

An optimal alignment is composed  
of optimal subalignments



## Result of Backtrace

- Two strings and their **alignment**:

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| I | N | T | E | * | N | T | I | O | N |
|   |   |   |   |   |   |   |   |   |   |
| * | E | X | E | C | U | T | I | O | N |





# Performance

- Time:

$O(nm)$

- Space:

$O(nm)$

- Backtrace

$O(n+m)$

# Minimum Edit Distance

# Weighted Minimum Edit Distance



# Weighted Edit Distance

- Why would we add weights to the computation?
  - Spell Correction: some letters are more likely to be mistyped than others
  - Biology: certain kinds of deletions or insertions are more likely than others





# Weighted Min Edit Distance

- Initialization:

$$D(0, 0) = 0$$

$$D(i, 0) = D(i-1, 0) + \text{del}[x(i)]; \quad 1 < i \leq N$$

$$D(0, j) = D(0, j-1) + \text{ins}[y(j)]; \quad 1 < j \leq M$$

- Recurrence Relation:

$$D(i, j) = \min \begin{cases} D(i-1, j) & + \text{del}[x(i)] \\ D(i, j-1) & + \text{ins}[y(j)] \\ D(i-1, j-1) & + \text{sub}[x(i), y(j)] \end{cases}$$

- Termination:

$D(N, M)$  is distance

